Report

Tentacrawl

Barnabas Tanczos

23 April 2025

1 Overview

The objective of this practical was to design and implement a video game in the Processing framework, including procedural generation techniques for creating levels. All four levels of the requirements have been implemented in a modular game engine. The game *Tentacrawl* demonstrates these techniques with a layered procedural dungeon generation algorithm.

2 Design

The primary design goal of the *Tentacrawl* game was to showcase detailed, interesting procedural levels that the player can navigate and interact with. The result is a roguelike dungeon crawler game with a series of algorithmically created levels populated with items to find and enemies to fight. Figure 1 shows a screenshot of the game in action. It has a simple, grid-based pixel art aesthetic using free online assets [1, 2] that captures a retro feel and provides an ideal framework for structured generation. The player is encouraged to explore the many room types and areas to find the required keys to progress and items to boost their octopus character.

In contrast to the previous practicals, the game is not focused on physics and movement, so combat takes place in turns. For every action the player takes, the nearby enemies take an action as well. This results in a slower-paced, tactical game, where moves must be anticipated and strategically countered. The system of charging and releasing attacks, in particular, forces the player to be deliberate with their decisions and plan ahead. As the difficulty progresses, both player and enemy weapon patterns become larger and more complex, thus reducing the margin of error. The game was inspired

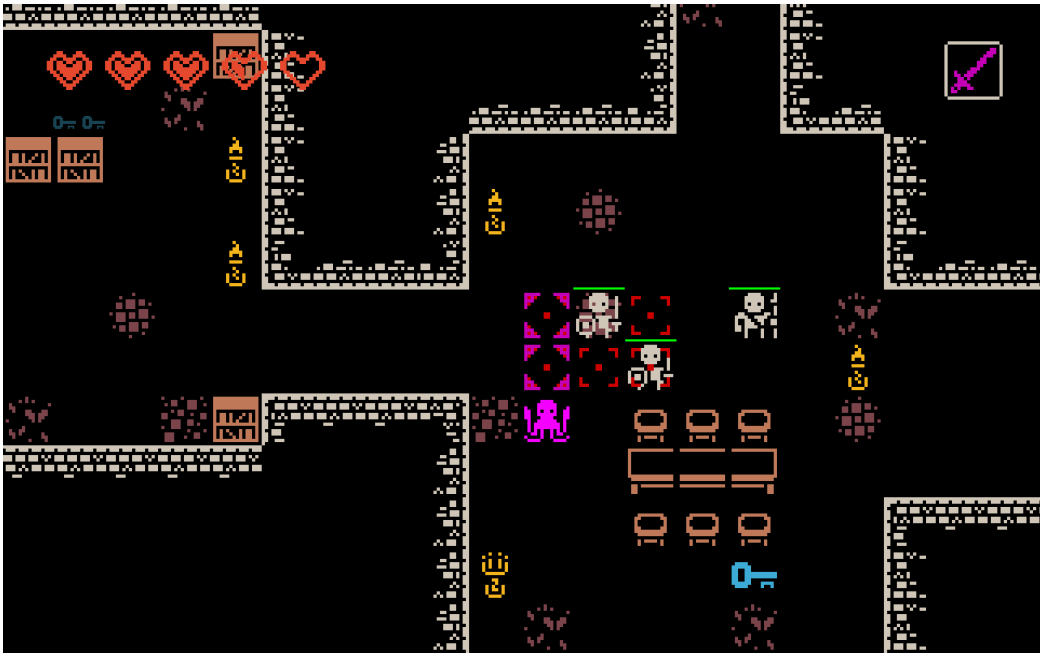


Figure 1: A screenshot from the *Tentacrawl* game, where the octopus protagonist fights a group of skeleton guards in the dining room.

by other minimalistic turn-based games such as *Crypt of the Necrodancer* (2015), *Into the Breach* (2018) and *Shogun Showdown* (2024).

3 Implementation

3.1 Game Engine

The game engine used is the same one developed throughout previous practicals using an entity component system (ECS). As it was already reasonably well-established, only a few minor changes were necessary for this project. The first of these was an update to the camera and UI rendering systems. Previously, these were entirely separate systems attached to the main game engine with their own behaviour parameters. In this version, these are replaced by a `CameraComponent` and `UIComponent` attached to a game object in each scene. This enables a smoother integration with other components, leveraging existing tools to manage the behaviour of these systems. For instance, since the camera now has a standard `TransformComponent`, it can be made to follow the player simply by adding a `KinematicSeekAI` from the previous game. Figure 2 shows an updated overview of the architecture.

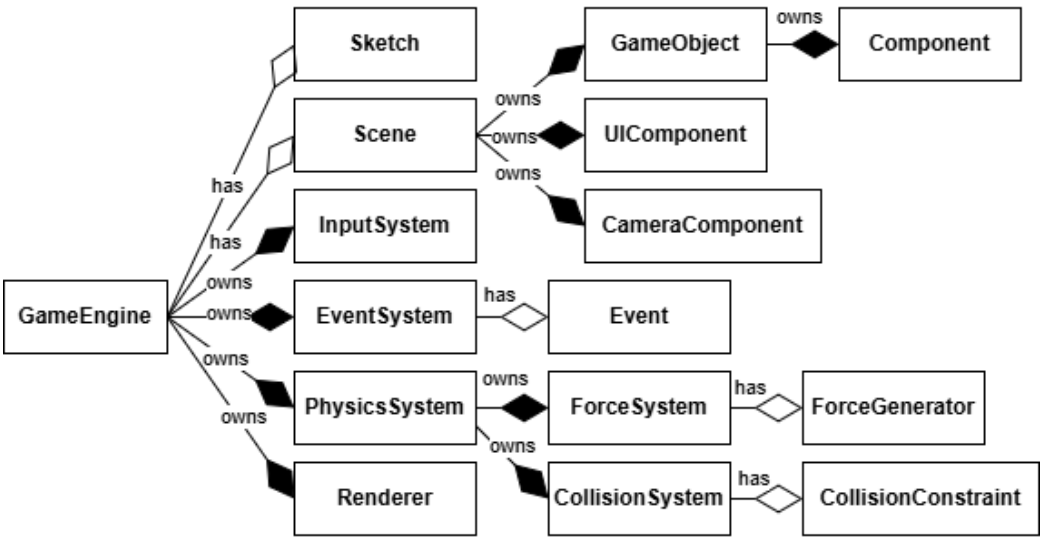


Figure 2: An updated UML diagram of the main relationships between the core game engine classes.

Another improvement was the addition of time-based events, which provide minimal functionality for delaying executable code to later frame updates. Each `TimedEvent` is updated every frame by the event system with the current time. Once its conditions are met, such as after a delay for `DelayedEvent` or within a certain duration for `RepeatedEvent`, the event invokes itself and triggers the defined behaviour in response. If it will not be needed again, the event is then removed from the system. Importantly, all behaviour is integrated within the game loop - rather than injected arbitrarily as with a separate thread - so it does not disrupt the update sequence by concurrent modification, for example.

3.2 Procedural Generation

The procedural aspect of the game involves the generation of levels. The `Map` class represents the level grid with a two-dimensional array of `MapSquare` objects. Each position on the map can be retrieved using a `Coord`, which is an integer version of a vector. Map squares, in turn, hold a background tile and a list of entities currently on the square. The background tiles are given by the `Tile` enum, which contains around 30 possible tiles. These may correspond to multiple actual images, chosen based on tile set rules concerning surrounding tiles or from a list of possible options with certain probabilities. The `MapDisplay` class is responsible for iterating through the squares and drawing the tiles to the camera. As the background does not

change, it is saved to a buffer that can be efficiently redrawn on every frame.

A new map is generated for each level by the `MapGenerator`, which coordinates with generators for the level structure, interior, and entities. First, the `StructureGenerator` creates a base map with a size and complexity determined by the difficulty. The map is divided into zones using binary space partitioning, as outlined by Shaker, Togelius and Nelson [3]. These are stored in a binary tree structure using `Node` instances. A root node encompassing the entire area is iteratively split, horizontally or vertically, until no area can be split further given a minimum area size constraint. Rooms can then be created inside each area, with a border to ensure that there are no overlaps. Subsequently, corridors are created between rooms. To ensure that the map is completely connected, it is sufficient to create one corridor between any left and right child of a non-leaf node. The algorithm prioritises connecting rooms that are not yet connected for a more interconnected map with fewer double corridors. For connecting two rooms, it calculates a random point on the sides of the rooms that are closest to each other. An L-shaped corridor is then formed from two rectangles to connect those points. Crucially, part of the corridor is marked to ensure no decorations are placed there and it can be traversed. Finally, the map is filled with wall tiles, and the rooms and corridors are carved out with ground tiles.

After the level structure is established, the decorative tiles are filled in by the `InteriorGenerator`. This makes use of the partition tree from the structure generator to divide rooms into groups, as seen in Figure 3. As children of the same node are guaranteed to be close to each other, the algorithm can simply label nodes of a sufficient depth with a `RoomGroup` enum, such as underground or residential. Based on the rooms defined under these groups, the leaves can then be labelled with a specific `Room` enum, such as mineshaft or library. Thus, the tree structure also enables a sensible decorative layout. To improve the diversity of room groups and rooms, these are taken from a shuffled queue rather than picked completely randomly. Every `Room` enum defines a density value that, together with the size of a room, is used to calculate the number of decorations. These can then be selected from the list of possible decorations for that room based on their defined weights. Each `Decoration` is a tile group template such as a tree or a table with chairs. It is placed in the room randomly, with a variable to determine whether this should be on the edges, in the centre or anywhere. Some may also be rotated for additional variety.

Lastly, entities are placed on the map by the `EntityGenerator`. The entrance where the player spawns is first defined. After calculating the distances from the entrance to each other node, the exit is then placed in one

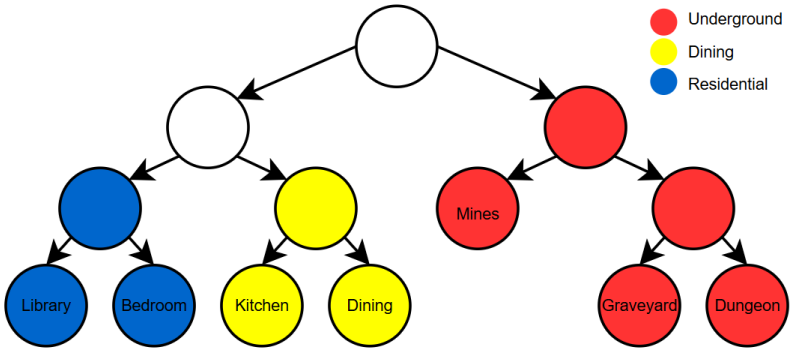


Figure 3: A diagram of the interior generation process, with rooms assigned to leaf nodes based on groups.

of the farthest, necessitating the most exploration to reach. A number of keys, ability pickups, and potions are also created in various other rooms depending on the difficulty, keeping a list of exclusions to avoid placing multiple important items in the same room. Finally, enemies are generated around the map to guard these locations. At higher difficulties, more powerful enemies are likely to generate, and weaker enemies are created in larger groups.

3.3 Gameplay

The main gameplay is comprised of relatively simple functionality. The `GameManager` is a singleton class that stores all information required between levels, such as player health and collected weapons. Each entity on the map, such as a key or potion, inherits from the `Entity` abstract class, which has a coordinate and methods to move it around the map. Movement may be blocked, however, if the square contains an entity or an impassable tile. Actors like the player and enemies further inherit from the `Creature` class. They also implement the `IDamageable` and `ITurnTaker` interfaces, which allow them to damage each other and take turns, respectively.

The player actions are all handled via events, responding to player input to move, charge or attack. The angle of attack is calculated from the vector from the player centre to the mouse. When an attack is activated, the target coordinates in that direction are highlighted based on the offsets given by ability type. Every other turn taker on the map then takes a turn, each delayed by a small duration using timed events to keep the action readable. Enemies in particular follow a simple behaviour of moving towards the player using a repurposed A* algorithm, then attacking once they are in range.

4 Evaluation

Overall, the design of the procedural level generation was a success. The binary space partitioning method was an effective solution for creating interconnected grid-based dungeons, especially to guarantee connectivity and group rooms based on theme. The three generators for structure, interior, and entities work together well, balancing playability, difficulty, and consistency with a good amount of variety. As a whole, the algorithms satisfy the project requirements. Regarding the *Tentacrawl* game itself, I believe it achieved its goal of a tactical experience with appealing, polished visuals as well as interesting enemies and weapons to experience. In my opinion, it has good potential to be extended into a replayable experience with more exploration variety and strategic mechanics.

However, given the time constraints, the amount of content on the gameplay side is somewhat insufficient. It would greatly improve the excitement of exploration if the rooms had more interesting loot, hazards, and interactable environmental features. Similarly, there is a lack of mechanical distinction between different enemies and weapons. The strategic aspect would benefit from more unique mechanics such as defensive abilities, mobility options, and status effects. Meanwhile, additional testing of the game would have fine-tuned the balance to provide an optimal difficulty curve and prevent unfair situations. Moreover, the enemy AI could be more complex to avoid clustering and aim their abilities more intelligently.

References

[1] Kenney. 1-Bit Pack, 2021. Accessed: 2025-04-20 [online] https://kenney.nl/assets/1-bit-pack.

[2] Storytype Studio. Fantasy Magist, 2023. Accessed: 2025-04-20 [online] https://www.dafont.com/fantasy-magist.font.

[3] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Springer, 2016.